CS3212 Algorithms

Aya Zirikly
10/20/2026

Midterm Exam Feedback

Quiz3 stats

Min 4.00 Max10.00 Average 8.07142

Grade Distribution

90-100:21

80-89:9

70-79:12

60-69:3

50-59:2

40-49:2

Pattern Search & String Matching

The String Search Problem (Definition)

Given two strings:

- **Text (T):** A long host text of length n (`T[0 ... n-1]`).
- **Pattern (P):** A shorter query string of length m (`P[0 ... m-1]`), where `m <= n`.

Both strings consist of characters drawn from a finite set called the **Alphabet (Σ Sigma)**.

Goal

Find all valid **shift positions s** (where `0 <= s <= n - m`) such that the pattern `P` occurs starting at index `s` in `T`.

A shift `s` is valid if:

T[s + j] == P[j] for all 0 <= j <= m - 1

Problem Variations & Output Types

Common Output Formats

Depending on the application, a string search algorithm may return:

1. **First Occurrence:** The smallest index s where P matches T (or -1 if not found).
2. **All Occurrences:** A list of all valid starting indices [s1, s2, ...].
3. **Count:** The total number of times P appears in T.

Examples

Text (T): "abacaabadcab" (n = 12)

Pattern (P): "aba" (m = 3)

Alphabet (Sigma): {'a', 'b', 'c', 'd'}

Matches found at shift indices:

- s = 0: T[0...2] = "aba" (Valid Match)
- s = 5: T[5...7] = "aba" (Valid Match)

Real-World Motivations & Applications

Why String Matching Matters

1. **Text Editors & Web Browsers:** Implementing Ctrl + F / Cmd + F functionality across large documents.
2. **Computational Biology (Bioinformatics):** Searching for specific DNA/RNA nucleotide sequences (e.g., matching a gene sequence of length m inside a genome of length n = 3 * 10^9 across Sigma = {A, C, G, T}).
3. **Plagiarism Detection & Spam Filtering:** Detecting exact or partial duplicate phrase matches in emails or submitted papers.
4. **Intrusion Detection Systems (IDS):** Scanning network packet payloads against known malicious signature patterns in real-time.

Baseline Approach: Naive (Brute-Force) String Matching

The Strategy

Test every possible shift position `s` from `0` to `n - m` one by one. For each shift `s`, compare characters of `P` against `T` from left to right until a mismatch occurs or the entire pattern matches.

```
NAIVE-STRING-MATCHER(T, P):
    n = length(T)
    m = length(P)

    if m > n or m == 0:
        return

    for s = 0 to n - m:
        j = 0
        while j < m and T[s + j] == P[j]:
            j = j + 1
        if j == m:
            print "Pattern occurs with shift" s
```

Analyzing the naive approach

Complexity Analysis

- **Best-Case Time:** `O(n)`
  - Occurs when the first character `P[0]` mismatches immediately at every shift `s` (e.g., `T = "aaaaaaaaaa"`, `P = "bbbb"`). We do only 1 comparison per shift.
- **Worst-Case Time:** `O((n - m + 1) * m)` which simplifies to **`O(n * m)`**

Why O((n - m + 1) * m)?

1. Where does (n - m + 1) come from?

The Outer Loop checks every possible starting position (shift s) where the pattern P can fit inside text T:

for s = 0 to n - m:

- First shift: `s = 0`
- Second shift: `s = 1`
- Last shift: `s = n - m`

To count how many total shifts (iterations) this loop runs, we use the standard formula for inclusive ranges:

Total Shifts = (End) - (Start) + 1 = (n - m) - 0 + 1 = n - m + 1

Why O((n - m + 1) * m)? *Step-by-Step Breakdown*

1. **Why does the loop end at n - m?**
    - Length of Text T = n (valid indices: 0 to n-1).
    - Length of Pattern P = m.
    - If you place P starting at index s, its last character aligns with index s + m - 1 in T.
    - The last valid index in T is n - 1.
    - Setting s + m - 1 = n - 1 gives s = n - m.
    - Thus, s = n - m is the absolute **last position** where P fits inside T.
2. **Why add 1?**
    - When counting how many integers are in an inclusive range [a, b], subtracting b - a gives the *difference*, which misses the starting item.
    - **Inclusive Counting Rule:** Count = End - Start + 1.

Why O((n - m + 1) * m)?

Example: If `T = "aaaaaaaaaa"` (`n = 10`) and `P = "aaab"` (`m = 4`):

- Shifts run from `s = 0` to `s = 10 - 4 = 6`.
- Total shifts evaluated: `6 - 0 + 1 = 7` shifts (`s = 0, 1, 2, 3, 4, 5, 6`).

Why O((n - m + 1) * m)?

2. Where does the * m come from?

For **each** of those (n - m + 1) shifts, the Inner Loop compares characters of P against T one by one:

```
while j < m and T[s + j] == P[j]:
    j = j + 1
```

In the worst-case scenario:

- You match m - 1 characters successfully.
- You fail (mismatch) on the very last character (j = m - 1).
- This forces the inner loop to do **m character comparisons** before giving up and moving to the next shift.

Total Character Comparisons = (Number of shifts) * (Comparisons per shift)

$$\text{Total Character Comparisons} = \underbrace{(n - m + 1)}_{\text{Number of shifts}} \times \underbrace{m}_{\text{Comparisons per shift}} = (\mathbf{n - m + 1}) \cdot \mathbf{m}$$

O((n - m + 1) * m) Example

3. Tracing the Example: T = "aaaaaaaaaa", P = "aaab"

- **Shift s = 0:**
  - Compares T[0], T[1], T[2] against P[0], P[1], P[2] -> **Matches** 'a', 'a', 'a' (3 checks).
  - Compares T[3] ('a') against P[3] ('b') -> **Fails!**
  - Total comparisons for shift 0: **4 comparisons** (m).
- **Shift s = 1:**
  - Resets back to T[1]. Compares T[1..4] against P[0..3] -> **Fails on 4th character!**
  - Total comparisons for shift 1: **4 comparisons** (m).

For n = 10 and m = 4:

Total Comparisons = (10 - 4 + 1) * 4 = 7 * 4 = 28 comparisons

Why does (n - m + 1) * m simplify to O(n * m)?

(n - m + 1) * m = n*m - m^2 + m

- Since m <= n, the term n * m dominates the growth rate.
- Lower-order terms (like +m) and subtracted terms (like -m^2) are dropped.
- Therefore, O(n*m - m^2 + m) simplifies cleanly to **O(n * m)**.

Analyzing the naive approach

Complexity Analysis

- **Best-Case Time:** `O(n)`
  - Occurs when the first character `P[0]` mismatches immediately at every shift `s` (e.g., `T = "aaaaaaaaaa"`, `P = "bbbb"`). We do only 1 comparison per shift.
- **Worst-Case Time:** `O((n - m + 1) * m)` which simplifies to **`O(n * m)`**

Why Naive Hitting Worst-Case is Wasteful

The Naive algorithm throws away information! When a mismatch occurs after partial matching, it resets back to index `s + 1` in `T` and re-inspects characters it has already seen.

Rabin-Karp Algorithm

Motivation

- Brute force compares up to m characters at every shift
- Idea: turn each length-m window into a single number, then compare numbers
- Comparing two numbers is O(1), so testing one shift (one position of P against T) takes O(1) instead of up to m character comparisons, as long as the numbers differ

Rabin-Karp Algorithm

Two problems to solve:

- Computing each window's number from scratch costs O(m) (fix: rolling hash)
- The numbers are too big to compare in O(1)
  - a pattern of m characters becomes an m-digit number (m = 1000 gives a 1000-digit number)
  - a computer only compares numbers that fit in one machine word (about 19 digits) in O(1)
  - fix: keep only the remainder mod a prime q, e.g. q = 13, so every value is between 0 and 12
  - cost: different windows can share a remainder (31415 mod 13 = 7 and 67399 mod 13 = 7)
  - equal remainders only mean "maybe a match"; we check the characters to be sure

Strings as numbers

- P is a string of characters ('3', '1', '4', '1', '5')
  - value(P) = 3·10^4 + 1·10^3 + 4·10^2 + 1·10 + 5 = 31415
- With letters, map each to a digit first (a = 1, b = 2, c = 3): value(cab) = 312
- Build value(P) in Θ(m), one digit at a time: multiply by 10, add the next digit 3 → 31 → 314 → 3141 → 31415
- value(P) has m digits, too big for long patterns, so we keep only its remainder mod q
- Hash value (fingerprint) of P: p = value(P) mod q. With q = 13: p = 31415 mod 13 = 7
- For each text window: t_s = value(T[s+1..s+m]) mod q
- t_s ≠ p: definitely no match at shift s. t_s = p: possible match, check the characters

Strings as numbers

P

3	1	4	1	5

Read P left to right. At each step: multiply by 10, add the next digit.

step	digit	exact value	same thing, mod 13 at every step
1	3	3	3
2	1	3·10 + 1 = 31	(3·10 + 1) mod 13 = 31 mod 13 = 5
3	4	31·10 + 4 = 314	(5·10 + 4) mod 13 = 54 mod 13 = 2
4	1	314·10 + 1 = 3141	(2·10 + 1) mod 13 = 21 mod 13 = 8
5	**5**	**3141·10 + 5 = 31415**	**(8·10 + 5) mod 13 = 85 mod 13 = 7**

Left: the exact number grows with every digit (m digits in the end).

Right: keep only the remainder after each step. Values stay below 13·10 = 130, and the result is the same: 31415 mod 13 = 7.

Fingerprint

The difference from the exact number is that the remainder is a fingerprint, not a unique ID. Many strings share the fingerprint 7 (31415 and 67399, for instance). So:

- different fingerprints mean the strings are definitely different, and you skip that window;
- the same fingerprint means the strings are probably equal, so you compare the characters to confirm.

Fingerprint

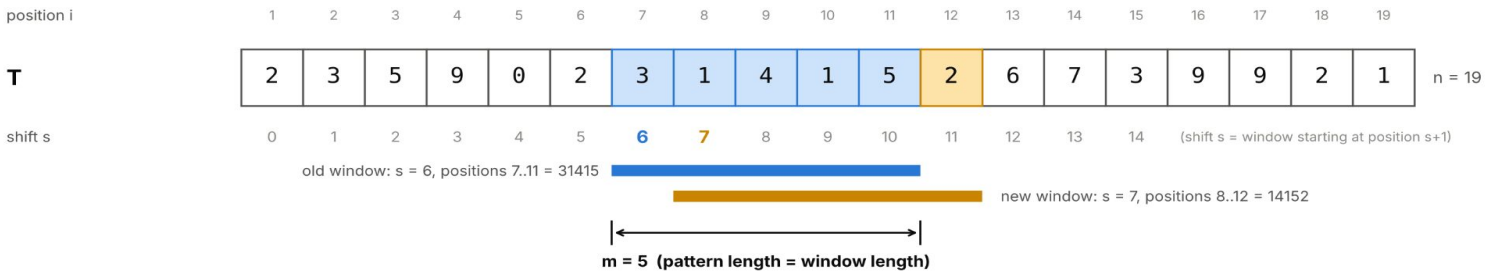
Two people with the same birthday month are not necessarily the same person, but two people with different birthday months are definitely different people.

The remainder works the same way: different remainder means definitely no match, same remainder means check.

Going from window s to window s+1:Rolling-hash

- drop the high-order digit, shift left, add the new low-order digit
- h = d^(m−1) mod q precomputed: $t_{s+1} = (d \cdot (t_s - h \cdot T[s+1]) + T[s+m+1]) \bmod q$
- Each update is O(1) => all n − m + 1 window values cost Θ(n − m + 1) after Θ(m) setup

Rolling-hash

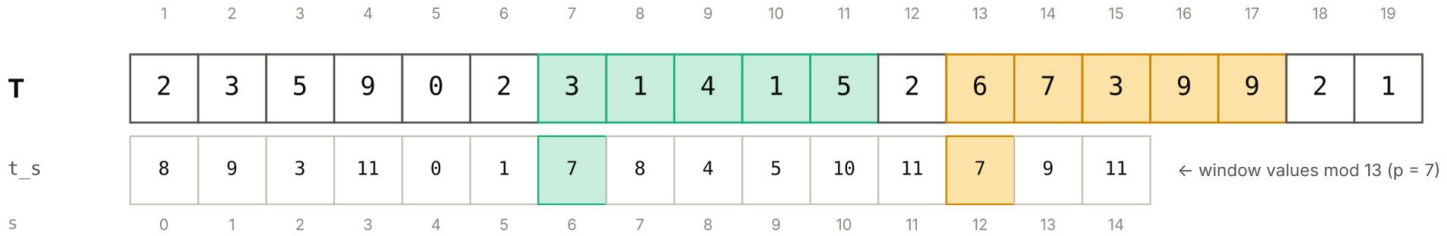



```
drop high digit   31415 − 3·10^(m−1) = 31415 − 3·10000 = 1415
shift left        1415 · 10      = 14150
add low digit     14150 + 2      = 14152
```

same thing on hash values (from s = 6 to s = 7, q = 13):

```
t_(s+1) = (d·(t_s − h·T[s+1]) + T[s+m+1]) mod q,   here s = 6, m = 5, d = 10, q = 13
    t_6 = 7 (hash of 31415)   h = d^(m−1) mod q = 10^4 mod 13 = 3
    T[s+1] = T[7] = 3 (digit leaving)   T[s+m+1] = T[12] = 2 (digit entering)
t_7 = (10·(7 − 3·3) + 2) mod 13 = (−18) mod 13 = 8
check: 14152 mod 13 = 8   ✓     O(1) work per shift
```

Same hash ≠ same string: valid matches vs. spurious hits”



s = 6: t_s = 7 = p, verify 31415 = 31415 → valid match ✓

s = 12: t_s = 7 = p, verify 67399 ≠ 31415 → spurious hit × (hash collision)

every other shift: t_s ≠ p, rejected after one number comparison, no character checks

Rabin-Karp pseudocode (CLRS)

```
RABIN-KARP-MATCHER(T, P, d, q)
  n = T.length
  m = P.length
  h = d^(m-1) mod q
  p = 0
  t_0 = 0
  for i = 1 to m                          // preprocessing, Θ(m)
      p   = (d*p   + P[i]) mod q
      t_0 = (d*t_0 + T[i]) mod q
  for s = 0 to n - m                        // matching
      if p == t_s                         // hash hit
          if P[1..m] == T[s+1..s+m]           // verify, Θ(m)
              print "Pattern occurs with shift" s
      if s < n - m
          t_(s+1) = (d*(t_s - T[s+1]*h) + T[s+m+1]) mod q
```

Time complexity

- Preprocessing: Θ(m) (compute h, p, t_0)
- Matching cost, in one line:

  total = (n − m + 1) · O(1) [hash check at every shift] + (number of hits) · Θ(m) [verify, only when t_s = p]

- The verify is inside an if, so the number of hits decides the case:
- Matching, worst case: Θ((n − m + 1) · m)
  - Every shift is a hit and must be verified, same as brute force
  - Example: T = a^n, P = a^m (every window is a valid match)

Time complexity

- Matching, best case: $\Theta(n - m + 1)$, so $\Theta(n + m)$ total
  - no window hashes to p: one O(1) number comparison per shift, no character checks
- Expected case (CLRS): if there are v valid shifts and q is a random prime ≥ m, expected spurious hits ≈ n/q, and the matching time is $O(n) + O(m \cdot (v + n/q))$
  - with v = O(1) and q ≥ m => v + n/q is dropped => O(n + m) expected

Best vs worst case

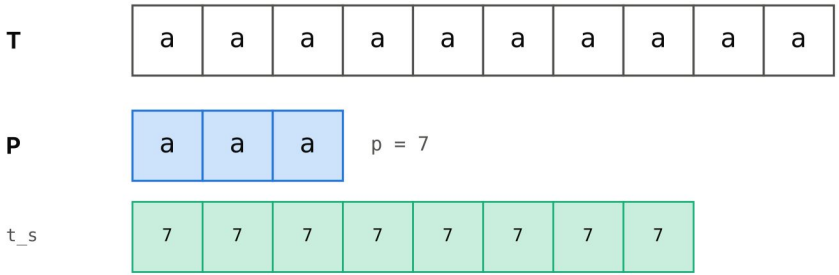
Worst: T = aaaaaaaaaa, P = aaa (with a = 1, d = 10, q = 13)

- p = value(aaa) mod 13 = 111 mod 13 = 7
- every window is also aaa, so every t_s = 7 = p: all 8 shifts are hits
- 8 shifts × 3 characters = 24 character checks

Best: T = 1111111111, P = 31415 (d = 10, q = 13)

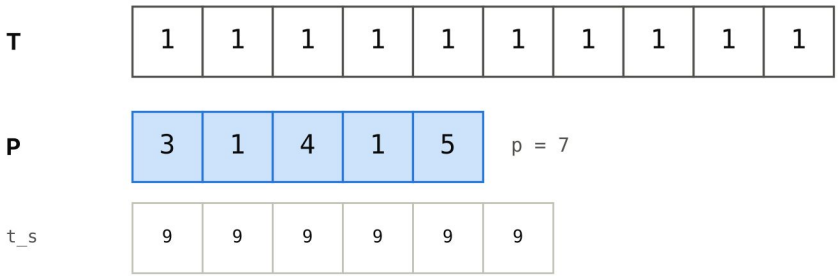
- p = 31415 mod 13 = 7
- every window of T is 11111, so t_s = 11111 mod 13 = 9
- 9 ≠ 7 at all 6 shifts: 6 number comparisons and zero character checks
- Lesson: Rabin-Karp's worst case is no better than brute force; its strength is the typical case with few hits

Worst case: T = aaaaaaaaaa, P = aaa (a = 1, d = 10, q = 13)



every window hashes to p → verify all m chars at all n−m+1 = 8 shifts → 8 × 3 = 24 char checks: Θ((n−m+1)·m)

Best case: T = 1111111111, P = 31415 (d = 10, q = 13)



no window hashes to p → one O(1) number comparison per shift, zero char checks → Θ(m) + Θ(n−m+1) = Θ(n + m)

Rabin-Karp takeaways

- Simple to implement: a few arithmetic operations per shift (two multiplies, a subtract, an add and a mod) plus a character-by-character check on hits; no tables or auxiliary arrays, $O(1)$ extra space
- Fast in practice: $O(n + m)$ expected when there are few matches and spurious hits
- Worst case $\Theta((n - m + 1) \cdot m)$, same as brute force, when almost every shift is a hit
- Pick q as a large prime to make spurious hits rare (about n/q of them)

Rabin-Karp takeaways

The hashing idea extends easily: search for many patterns at once (store all pattern hashes in a set), 2D pattern matching, plagiarism detection

1. Compute the hash of each pattern and put the hashes in a set (a hash table). For example, with q = 13, the patterns 31415 and 92653 have hashes 7 and 2, so the set is {7, 2}.
2. Slide over T once with the rolling hash.
3. At each shift, instead of asking “is t_s = p?”, ask “is t_s in the set?”. That’s still O(1) with a hash table.
4. If hit, verify the characters against the pattern(s) with that hash.